

Step-by-Step Explanation

Method 1: create_sessions_df()

Purpose

Creates a Pandas DataFrame from raw charging session records.

Input

```
[  
[501,"ST01","Bengaluru","2025-02-01",42.5,75,"Paid"]  
]
```

Output

Session ID	StationID	City	ChargingDate	EnergyKWh	DurationMinutes	PaymentStatus
501	ST01	Bengaluru	2025-02-01	42.5	75	Paid

Method 2: clean_sessions_data()

Rules

Remove rows if:

- SessionID is null
- StationID is null
- City is null
- PaymentStatus is null
- EnergyKWh ≤ 0
- DurationMinutes ≤ 0
- Invalid payment status

Valid statuses:

```
["Paid","Pending","Failed"]
```

Example

Before:

EnergyKWh	DurationMinutes
45	60
-10	70
20	0

After:

EnergyKWh	DurationMinutes
45	60

Method 3: add_long_session_flag()

Rule

DurationMinutes > threshold

If True:

1

Else:

0

Example:

Threshold = 90

Duration

75

130

Output:

Duration	Flag
----------	------

75	0
----	---

130	1
-----	---

Method 4: station_utilization_summary()

Group By

StationID

Calculate

count()

sum()

mean()

Example

StationID	Energy
-----------	--------

ST01	42.5
------	------

ST01	55
------	----

Result

StationID	SessionCount	TotalEnergyKWh	AverageDuration
-----------	--------------	----------------	-----------------

ST01	2	97.5	102.5
------	---	------	-------

Method 5: high_energy_stations()

Logic

TotalEnergyKWh > threshold

Important:

>

NOT

>=

Hidden Test Case checks this.

Example:

Threshold = 80

Station	Energy
---------	--------

ST01	97.5
------	------

ST02	80
------	----

Output:

Station

ST01

ST02 excluded because energy equals threshold.

Method 6: city_revenue_summary()

Revenue Formula

Revenue = EnergyKWh * 18

Only Paid sessions contribute.

Example

City	Energy	Status
Bengaluru	42.5	Paid
Bengaluru	55	Pending

Revenue

$42.5 * 18 = 765$

Output

City	Revenue
Bengaluru	765

Pandas Cheat Sheet Used in This Problem

Create DataFrame

```
pd.DataFrame(data, columns=columns)
```

Remove Nulls

```
df.dropna(subset=["col1", "col2"])
```

Filter Rows

```
df[df["EnergyKWh"] > 0]
```

Multiple Conditions

```
df[(df["EnergyKWh"] > 0) &
   (df["DurationMinutes"] > 0)]
```

Check Values in List

```
df["PaymentStatus"].isin(
    ["Paid", "Pending", "Failed"]
)
```

Create New Column

```
df["Revenue"] = df["EnergyKWh"] * 18
```

Convert Boolean to Integer

`(condition).astype(int)`

Example

`(df["DurationMinutes"] > 90).astype(int)`

Group By Aggregation

```
df.groupby("StationID").agg(  
    Total=("EnergyKWh", "sum")  
)
```

Count Records

`count()`

Sum Values

`sum()`

Average

`mean()`

Rename Columns

```
.rename(  
    columns={  
        "EnergyKWh": "TotalEnergyKWh"  
    }  
)
```

Reset Index

`.reset_index()`

Time Complexity

Method	Complexity
<code>create_sessions_df</code>	$O(n)$
<code>clean_sessions_data</code>	$O(n)$
<code>add_long_session_flag</code>	$O(n)$
<code>station_utilization_summary</code>	$O(n)$
<code>high_energy_stations</code>	$O(n)$
<code>city_revenue_summary</code>	$O(n)$

where **n = number of charging session records**.

Solution

```
import pandas as pd
```

```
class EVChargingAnalyzer:
```

```
    # 1. Create Charging Sessions DataFrame
```

```
    def create_sessions_df(self, session_data: list) -> pd.DataFrame:
```

```
        columns = [  
            "SessionID",  
            "StationID",
```

```

    "City",
    "ChargingDate",
    "EnergyKWh",
    "DurationMinutes",
    "PaymentStatus"
]

```

```

return pd.DataFrame(session_data, columns=columns)

```

2. Clean Charging Sessions

```

def clean_sessions_data(self, df: pd.DataFrame) -> pd.DataFrame:

```

```

    # Remove null values

```

```

    df = df.dropna(
        subset=[
            "SessionID",
            "StationID",
            "City",
            "PaymentStatus"
        ]
    )

```

```

    # Keep positive energy and duration

```

```

    df = df[
        (df["EnergyKWh"] > 0) &
        (df["DurationMinutes"] > 0)
    ]

```

```

    # Valid payment statuses

```

```

    valid_status = ["Paid", "Pending", "Failed"]

```

```

    df = df[df["PaymentStatus"].isin(valid_status)]

```

```

    return df.reset_index(drop=True)

```

3. Add Long Session Flag

```

def add_long_session_flag(
    self,
    df: pd.DataFrame,
    duration_threshold: int
) -> pd.DataFrame:

```

```

    df = df.copy()

```



```

df["IsLongSession"] = (
    df["DurationMinutes"] > duration_threshold
).astype(int)

```

```

return df

```

4. Station Utilization Summary

```

def station_utilization_summary(

```

```

    self,

```

```

    df: pd.DataFrame

```

```

) -> pd.DataFrame:

```

```

    summary = (

```

```

        df.groupby("StationID")

```

```

        .agg(

```

```

            SessionCount=("SessionID", "count"),

```

```

            TotalEnergyKWh=("EnergyKWh", "sum"),

```

```

            AverageDuration=("DurationMinutes", "mean")

```

```

        )

```

```

        .reset_index()

```

```

    )

```

```

    summary["AverageDuration"] = (

```

```

        summary["AverageDuration"].round(1)

```

```

    )

```

```

    return summary

```

5. High Energy Stations

```

def high_energy_stations(

```

```

    self,

```

```

    df: pd.DataFrame,

```

```

    energy_threshold: float

```

```

) -> pd.DataFrame:

```

```

    result = (

```

```

        df.groupby("StationID")["EnergyKWh"]

```

```

        .sum()

```

```

        .reset_index()

```

```

        .rename(

```

```

            columns={"EnergyKWh": "TotalEnergyKWh"}

```

```

        )

```

```

    )

```

```

        return result[
            result["TotalEnergyKWh"] > energy_threshold
        ].reset_index(drop=True)

# 6. City Revenue Summary
def city_revenue_summary(
    self,
    df: pd.DataFrame
) -> pd.DataFrame:

    paid_df = df[
        df["PaymentStatus"] == "Paid"
    ].copy()

    paid_df["Revenue"] = (
        paid_df["EnergyKWh"] * 18
    )

    revenue_summary = (
        paid_df.groupby("City")["Revenue"]
        .sum()
        .reset_index()
    )

    return revenue_summary

# -----
# Sample Execution
# -----

analyzer = EVChargingAnalyzer()

data = [
    [501, "ST01", "Bengaluru", "2025-02-01", 42.5, 75, "Paid"],
    [502, "ST02", "Chennai", "2025-02-01", 28.0, 50, "Paid"],
    [503, "ST01", "Bengaluru", "2025-02-02", 55.0, 130, "Pending"]
]

df = analyzer.create_sessions_df(data)

print("Original Data")
print(df)

```

```
df = analyzer.clean_sessions_data(df)

print("\nCleaned Data")
print(df)

print("\nLong Session Flag")
print(analyzer.add_long_session_flag(df, 90))

print("\nStation Utilization")
print(analyzer.station_utilization_summary(df))

print("\nHigh Energy Stations")
print(analyzer.high_energy_stations(df, 80))

print("\nCity Revenue Summary")
print(analyzer.city_revenue_summary(df))
```